

Image Bundling Revisited: Atlasing Small Web Images under Modern Codecs and Protocols

Alexander Apartsin

Technical Report · 2026

ABSTRACT

Web pages load tens to hundreds of small images, and the median mobile page carries 13. Serving each one separately pays three costs that HTTP/2 multiplexing does not remove: per-request overhead, a fixed per-file structural cost (roughly 600 bytes for a JPEG), and the loss of cross-image redundancy. We measure how much of this bundling many images into one atlas recovers, for the three codecs that carry most web images and decode in every browser: JPEG, PNG, and WebP.

At matched per-tile quality, atlasing small tiles saves up to 26% of bytes under JPEG and up to 15% under WebP, independent of protocol. The saving is set by the ratio of per-file overhead to content size, so it falls as tiles grow: for photographs it drops from about 30% at 56 pixels to a few percent at 224 pixels, where WebP even turns slightly negative. Lossless formats lose under a naive grid but win 40–95% under codec-aware packing (vertical strips, shared palettes), so the question is how to pack, not whether. We distil the measurements into a coupling account, savings come from sharing fixed costs and losses from sharing adaptive state, and into an open-source tool that turns a directory of images into deployable bundles and matches an offline oracle on five unseen collections. A supporting browser study shows the byte savings also cut load time on HTTP/1.1 and HTTP/3.

1 Introduction

Product grids, icon sets, avatars, and decorative elements make small images the most numerous resource class on commercial pages. Bundling them into one image, the CSS sprite technique, was standard practice in the HTTP/1.1 era and fell out of favor when HTTP/2 multiplexing removed the per-connection request bottleneck. That reasoning addressed only the request-count cost. Two byte-level costs survive multiplexing untouched: every image file carries a fixed container overhead (headers, quantization tables, ISOBMFF boxes), and every file boundary prevents the codec from sharing entropy context, palettes, and predictors across images.

The study is deliberately scoped to the popular, universally-supported compressed formats: JPEG, PNG, and WebP together carry over 70% of images served on the web (HTTP Archive 2024: JPEG 32.4%, PNG 28.4%, WebP 12.0%), decode in every browser, and are where the practical savings live. The three formats price the per-file costs very differently: a JPEG file carries roughly 600 bytes of Huffman and quantization table definitions plus headers, a PNG a 67-byte structural floor plus per-scanline filter adaptation, and a WebP as little as 30 bytes of container around a heavily image-adapted VP8 payload. This report quantifies what bundling recovers, per codec, tile size, and image count, under a matched-quality protocol, and describes a testbed that measures the timing consequences under HTTP/1.1, HTTP/2, and HTTP/3.

The regimes this study measures map onto concrete page types. Product grids and image-search results serve 100–280-pixel photographic thumbnails by the dozens; recommendation strips, cart previews, and avatar rows serve 48–96-pixel photos; emoji and reaction pickers serve 24–64-pixel flat art by the hundreds, and are one place where sprite atlases remain in production use today (chat

applications ship emoji sheets; video platforms ship hover-preview storyboards as frame mosaics). The tile-size axis of the experiments spans exactly this range, so each regime can read its expected saving off the measured curves.

The display side needs no special machinery: a bundled tile is shown by any of CSS `background-position` (universal), `object-view-box` (Chromium), canvas `drawImage` region blits, or SVG `viewBox` cropping. The browser decodes the atlas once and paints windows into it.

Contributions. This report contributes (i) a matched-quality sweep of image atlasing across the three universally-supported web codecs (JPEG, PNG, WebP) over tile size and image count, establishing that the byte saving is governed by the ratio of per-file structural cost to content bytes, and to our knowledge the first peer-reviewed measurement of image-atlasing byte savings resolved by codec and tile size at matched perceptual quality (the one prior peer-reviewed sprite study [33] optimized PNG packing geometry and held the codec fixed); (ii) a network study of 2,515 validated cold browser loads that separates the timing effect of bundling across HTTP/1.1, HTTP/2, and HTTP/3 under emulated network conditions; (iii) a set of construction techniques with measured effect, PNG vertical-strip packing, explicit and LZ-window duplicate exploitation, chunking for bounded cache invalidation, delta updates via compression dictionaries, and encoder-parameter tuning that flips WebP's photographic-atlas penalty to a gain; (iv) a coupling-spectrum account that unifies the results, savings arise from sharing fixed costs and losses from sharing adaptive state; and (v) an open-source construction heuristic that emits deployable bundles from a directory of images.

2 Related work

2.1 Resource bundling and image spriting

Combining many small resources into one is long-standing web practice, catalogued in the performance-engineering literature (Souders [35]) as spriting, concatenation, and DataURI inlining, all of which trade requests for cache granularity in the HTTP/1.1 era; CSS sprites [10] are the image-specific form. The regime they address is real and growing: measurement of page composition shows the modern page is dominated by many small, heterogeneous resources (Butkiewicz et al. [34]), and image requests are the largest class (HTTP Archive [8]). Whether bundling still pays under HTTP/2 multiplexing has been examined mainly for JavaScript and CSS: Khan Academy found unbundled JS slower than bundles under HTTP/2, attributing the gap to worse per-file compression [1], and practitioner analyses reach the same conclusion for concatenated assets [2, 11]. Modern build tools encode the trade-off directly as an inlining size threshold (webpack asset modules, Vite's asset limit). The closest academic work on the packaging question is Marx et al. [29], who test concatenation, embedding, and sharding under HTTP/2 and find the HTTP/1 packaging habits still often help; broader page-load studies show load time is governed by resource dependencies and compute, not bytes alone (WProf [26], How Speedy is SPDY? [27], Polaris [28]), so request-count reductions are only part of the story. The one peer-reviewed study of image spriting itself is Marszalkowski et al. [33], who formulate CSS-sprite construction as a geometric packing problem, model load time, and measure how a PNG sprite's aspect ratio affects its file size, holding the codec fixed at PNG and optimizing layout area. Their axis is packing geometry; ours is codec and tile size at matched quality, which they do not vary. A CSS-Tricks case study reports a 223-icon sprite at roughly 10 KB versus 115 KB unbundled [2], but without protocol-level timing. We find no published measurement that quantifies image atlasing under HTTP/2 or HTTP/3, nor a codec-by-tile-size atlasing sweep at matched quality.

2.2 Protocol performance: HTTP/2 and HTTP/3

HTTP/2 multiplexes many requests over one connection [12] but suffers transport-level head-of-line blocking under loss; HTTP/3 over QUIC [13] removes it with per-stream recovery. Whether the newer protocol is faster in practice is contested and configuration-sensitive: de Saxcé et al. [36] found HTTP/2 not uniformly faster than HTTP/1.1, and rigorous QUIC evaluation shows performance swings widely with implementation and workload (Kakhki et al. [37]). Measurement studies characterize HTTP/2 adoption and page-load behavior [3] and server push [4], and both controlled benchmarks [5] and adoption studies [14] report HTTP/3 gaining most under packet loss while sitting near parity at zero loss. Domain sharding, the historical technique of spreading resources across hosts to widen HTTP/1.1 concurrency [15], is the opposite of bundling; recent measurement finds such HTTP/1-era habits persist even under HTTP/2 [30], which motivates our condition set. This body of work establishes that protocol-level outcomes for a given payload are as much a property of the deployment as of the standard, the context in which our own HTTP/3 concurrency observation (Section 5.5) should be read; none of it isolates a many-small-images payload against a bundled baseline.

2.3 Small-image coding and multi-image containers

The fixed per-file cost each codec carries is documented through minimal one-pixel files: JPEG XL 24 B, WebP 30 B, PNG 67 B, JPEG 155 B, AVIF 303 B [6, 7]. This overhead makes small images the worst case for heavyweight containers, and production pipelines act on it: Cloudinary declines AVIF for images below 5,000 pixels because the box overhead outweighs the coding gain [16]. Several formats already provide intermediate ways to share structure across many images without a full pixel atlas: JPEG's abbreviated format carries one table-specification datastream ahead of table-less scans [17]; animated WebP stores independently-coded frames in one container [19]; APNG and MNG [40] and the HEIF image-collection format [41] package multiple images in one resource. These occupy the low-coupling end of the spectrum this report maps, sharing container and tables but not the coding model. The JPEG [17], PNG [18], and WebP [19] format definitions specify the table, chunk, and container structures our measurements amortize; JPEG XL adds a modern architecture aimed partly at small images [38]. Matched-quality byte comparison across these formats is established for single images (Google's WebP study measures WebP against JPEG and PNG at equal SSIM [39]) and extended by peer-reviewed rate-distortion studies against newer codecs [31]; we carry the same matched-quality discipline to the atlas-versus-individual question. Perceptual image-quality assessment, on which our matched-quality protocol rests, is grounded in the structural similarity index [25]. We are not aware of prior work that measures how bundling recovers per-file overhead as a function of codec and tile size.

2.4 Texture atlases and sprite packing

Packing many images into one is standard in real-time graphics, where texture atlases [9, 20, 32] and skyline/MaxRects bin-packing [21] pursue a different objective: production atlas tools (TexturePacker, engine sprite packers) and virtual-texturing systems minimize GPU state changes and packed area, and tune padding for mip-sampling and filtering correctness at tile boundaries rather than for encoded size. The optimization target is binding count and texture memory, not transmitted bytes, so this literature does not subsume a delivery-oriented study: our objective is encoded

size and adaptive codec state under matched perceptual quality, which area-minimizing packers neither measure nor optimize.

2.5 Cache granularity and delta delivery

Bundling trades cache granularity for fewer requests, a tension delivery research has long addressed. HTTP delta encoding transmits only the difference between a cached and a current resource (RFC 3229 [42]) using generic differencing formats (VCDIFF, RFC 3284 [43]); shared-dictionary schemes reuse previously delivered bytes as a dictionary for later responses (SDCH [44]), the idea that shared-window and trained-dictionary compressors (Brotli [22], zstd [23]) generalize and that Compression Dictionary Transport [24] revives for the modern web. Standard HTTP caching semantics (RFC 9111 [45]) fix the granularity at the resource, which is exactly what bundling coarsens; we apply delta and dictionary delivery to the whole-bundle cache-invalidation problem (Section 6.2). HTTP Archive's Web Almanac [8] supplies the population statistics (median 13 images per mobile page, format share) that make the many-small-images regime the common case rather than an edge case.

3 The atlas serving model

3.1 Rendering a tile from an atlas in HTML

A bundled page references one image resource and displays each tile by cropping a window into it. Four standard mechanisms cover every deployment context. The classic and universal one positions the atlas behind a fixed-size element as a background:

```
<div class="tile" style="background-image:url(atlas.webp);  
    background-position:-144px -216px"></div>  
/* .tile has width:72px; height:72px */
```

The element shows the 72x72 region whose top-left corner sits at (144, 216); `background-size` rescales the whole atlas when display size differs from stored size. Chromium additionally supports cropping a real `<img>` element, which preserves alt text, native lazy loading, and `fetchpriority`:

```

```

Where a real image element is needed cross-browser, a fixed-size wrapper with `overflow:hidden` around a negatively-offset `<img>` gives the same window, and SVG gives it declaratively (`<svg viewBox="144 216 72 72"><image href="atlas.webp"/></svg>`). Canvas completes the set for programmatic UIs: `ctx.drawImage(atlas, 144, 216, 72, 72, dx, dy, 72, 72)` blits any region after a single decode. In every mechanism the browser fetches and decodes the atlas once, holds one decoded copy, and paints windows into it; per-tile cost is a paint, not a decode. The experiments use `background-position`, the mechanism with universal support.

3.2 Delivery, caching, and memory

An atlas changes the unit of caching from the tile to the bundle. With standard immutable content-addressed URLs (`atlas.3fe2a1.webp`, `Cache-Control: immutable`), an unchanged atlas costs zero requests on a warm cache, and the decode-once property still applies. The cost appears on content change:

editing one tile invalidates the whole bundle, so the expected re-download per deploy grows with bundle size. Splitting the collection into k chunked atlases bounds the worst-case invalidation at $1/k$ of the collection while retaining nearly all of the byte and request savings (the ordering study of Section 5.3 finds four chunks price within about a percentage point of one atlas), which makes chunking the practical default for collections that update piecemeal. Grouping tiles by update cadence (stable icon set in one chunk, weekly seasonal art in another) further confines invalidation to the chunk that actually changed.

The second resource to budget is decoded memory: a decoded atlas occupies width $\times$ height $\times$ 4 bytes regardless of its encoded size, so a 4096x4096 atlas holds 64 MB of RGBA for a 300 KB transfer. Individual images decode lazily and can be evicted per tile; an atlas is decoded and resident as a unit while any tile is visible. Chunking bounds this cost the same way it bounds invalidation, and below-the-fold chunks combine with lazy loading so off-screen tiles cost neither bytes nor memory.

4 Methodology

4.1 Assets and packing

Two deterministic asset classes anchor the study: 521 Twemoji 72x72 flat-art tiles (alpha composited over white) and 521 photographic 224x224 thumbnails, with the photo set additionally downscaled to 112 and 56 pixels for the size sweep and synthetic icon, product-thumbnail, and avatar corpora added for the use-case tests (Section 5.4). Tiles are packed row-major into a near-square grid; a padding variant edge-replicates each tile by 8 or 16 pixels, and a vertical-strip variant packs one tile per row band. All conditions consume identical source pixels.

4.2 Codecs and matched-quality protocol

Following the study's scope, the three dominant web formats encode each condition: libjpeg-turbo JPEG, WebP (lossy and lossless), and PNG, with the lossy codecs swept over a quality ladder $q \in \{30, 50, 65, 80, 90\}$. Quality is the mean over tiles of the per-tile luma SSIM [25], each tile scored after cropping it back out of the decoded artifact so atlas border bleed is charged to the atlas; the matched target of 0.97 is therefore a mean-tile floor, and Section 5.4 reports the per-tile spread where it is load-bearing. Bytes are compared at equal quality by log-linear interpolation of each condition's rate-distortion curve at the target; the five-point ladder is coarse, so where the atlas and individual curves nearly coincide the interpolation is unstable and equal-quality byte comparison is used instead (Section 5.4). When an atlas's entire ladder exceeds the target, its cheapest measured point is used, understating the atlas's advantage, and the value is reported as a lower bound. Three invariants validate the harness: lossless conditions score SSIM exactly 1.0; an atlas of one image is byte-identical to the individual file; padding never reduces atlas bytes.

4.3 Network testbed

Byte savings are protocol-independent; timing effects are not. The network testbed serves every condition from a Caddy server inside WSL2 with three protocol endpoints (HTTP/1.1, HTTP/2, HTTP/3 on QUIC), shapes real packets with `tc netem` (delay, bandwidth, random loss applied on the server egress), and drives a fresh cold-cache Chromium instance per page load via Playwright. Each page stamps a timestamp after every tile is decoded and two animation frames have painted, giving a single time-to-all-tiles-visible endpoint, and records per-resource transfer sizes and the negotiated protocol

from the Resource Timing API; every load is validated against the intended protocol and against the manifest's byte totals, so a page that fetched the wrong way cannot enter the dataset. Four serving conditions run: N individual files, one atlas, four chunked atlases, and a byte-bundle (the N encoded files concatenated into one binary resource plus an offset index; the client slices the buffer and decodes each tile from its own bytes, retaining per-file codec adaptation while collapsing N requests into one). The sweep crosses these with $N \in \{50, 200, 500\}$, both asset classes as WebP q80, three protocols, and four network profiles. Within each profile the load order is randomized across conditions, protocols, and repetitions, and the first repetition of each cell is discarded as a warm-up, leaving 11 measured loads per cell.

4.4 Artifact availability

All code, asset manifests, raw per-run measurements, and the construction heuristic are released at github.com/ApartsinProjects/ImageBundling, and every table and figure in this report is regenerated from that data by a single build command. The measurements use Pillow 12.2 (libwebp 1.6.0, libjpeg-turbo via libjpeg 8.0, zlib-ng 1.3.1), Python 3.14, zstandard and Brotli for the delta-update study, Caddy 2.11.4 for the three-protocol server, and Playwright 1.58 driving Chromium 145 for the network loads. The photographic corpus is drawn from Lorem Picsum and the flat-art corpus from the Twemoji set; both frozen manifests are in the repository so every condition consumes identical source pixels.

5 Results

5.1 Byte savings at matched quality



Figure 1. A 100-tile atlas (10x10 grid of 72x72px tiles, 720x720px). Encoded with identical encoder settings both ways (distinct from Table 1’s matched-quality protocol), the 100 tiles cost 147,361 bytes as separate JPEG files and 112,059 bytes as this single JPEG (24% less), while 100 HTTP requests become one. Each tile is displayed with CSS background-position.

Table 1 reports the saving from atlasing at SSIM 0.97 across both classes. Three regularities organize the table. First, savings scale with the ratio of per-file structural cost to content bytes: 72-pixel flat-art tiles encode to 1–3 KB, so JPEG’s roughly 600 bytes of per-file tables and headers alone account for most of its measured 19–26% saving across N, while WebP’s 30-byte floor leaves it the smallest lossy gain (8–15%). Second, savings broadly increase with N and flatten by a few hundred tiles as amortization completes (the trend is not strictly monotonic; because each N draws one deterministic subset, small non-monotonicities such as WebP flat art at 8.3/15.2/8.2% for N = 50/200/500 reflect subset composition, not a scaling law). Third, tile size, not image count, is the dominant variable, and the photo rows of Table 1 trace the full crossover: downscaling the same 500 photographs from 224 to 112 to 56 pixels moves the JPEG saving from 3.0% to 9.8% to 29.8%, and moves WebP from –8.5% through –1.4% to +15.3%, placing WebP’s bundling break-even near 100-pixel tiles. Search-result and

recommendation thumbnails (48–120 px) therefore sit squarely in the paying regime for both formats, while product-grid images (200 px and up) pay only under JPEG, and only a few percent.

To confirm the crossover is not an artifact of the single deterministic subset each Table 1 cell uses, we resampled it: for each (tile size, N, codec) we drew 20 random N-tile subsets from the full pool and computed the matched-quality saving of each. The medians track Table 1 and the 95% bootstrap intervals are tight and separate the regimes. For photos the JPEG saving is 29.1% (95% CI [28.4, 29.5]) at 56 px, 9.3% [8.9, 9.8] at 112 px, and 2.6% [2.4, 2.8] at 224 px, all clearly positive; WebP is 14.8% [10.0, 17.4] at 56 px, straddles zero at 112 px ([−1.6, 1.9]), and is clearly negative at 224 px (−8.0% [−12.0, −6.7]). The WebP break-even near 100 px is thus the point where its interval crosses zero, not a single-sample coincidence.

Table 1. Bytes saved by atlasing (%) at matched per-tile SSIM 0.97 (lossy codecs; interpolated on the quality ladder) or exact lossless comparison (-ll columns), grid packing, no padding. Negative values (red) mean the atlas is larger.

class	N	JPEG	WebP	PNG	WebP-lossless
flat art 72px	10	19.4	—	4.3	-0.2
flat art 72px	50	26.3	8.3	5.4	5.4
flat art 72px	200	25.1	15.2	-3.1	-4.7
flat art 72px	500	26.3	8.2	-5.7	-3.6
photos 56px	10	22.3	15.3	-1.4	2.9
photos 56px	50	27.7	14.2	1.1	2.5
photos 56px	200	29.3	18.8	0.4	3.7
photos 56px	500	29.8	15.3	0.8	4.9
photos 112px	10	6.3	2.7	-3.8	1.8
photos 112px	50	8.4	1.7	-0.9	4.3
photos 112px	200	9.4	-0.1	-1.8	4.3
photos 112px	500	9.8	-1.4	-1.6	1.4
photos 224px	10	1.0	-4.3	-5.7	-3.4
photos 224px	50	1.8	-5.1	-2.8	3.0
photos 224px	200	2.6	-8.2	-3.8	0.3
photos 224px	500	3.0	-8.5	-3.6	-4.9

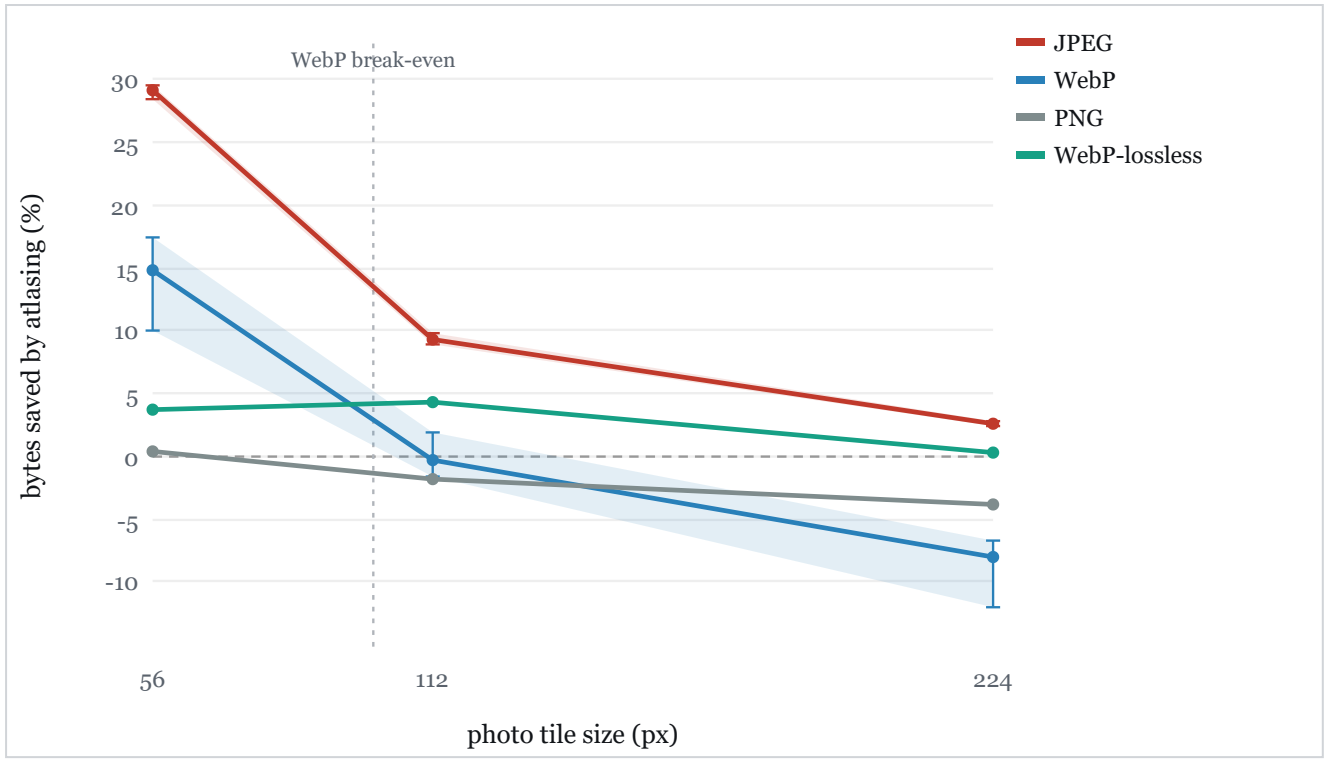


Figure 2. Byte savings from atlasing 200 photographic tiles at matched quality, as tile size grows from 56 to 224 pixels. JPEG and WebP are plotted as medians of 20 random subsets with 95% bootstrap confidence intervals (whiskers and shaded band); the lossless formats are exact. The saving tracks the per-file-overhead-to-content ratio and so falls with size: JPEG stays clearly positive throughout, while WebP's interval crosses zero near 100 px (dashed marker) and is clearly negative by 224 px. Table 1 gives the full grid over image count and the flat-art class.

Atlasing is not free where per-image adaptation matters. On flat art, both lossless formats lose from atlasing at $N \geq 200$ (PNG -3 to -6% , lossless WebP -4 to -5%): PNG chooses its prediction filter per scanline and an atlas scanline crosses dozens of unrelated tiles, while lossless WebP fits transforms and entropy codes per image, and one global model over hundreds of heterogeneous tiles cannot match hundreds of specialized ones. Lossy WebP shows the same inversion on photographic tiles: VP8 adapts entropy tables per image and allows at most four quantizer segments per frame, so an atlas of 500 diverse photos shares four segments where individual files had four each. This penalty is a quality effect, not a byte cost, and the -8.5% headline is best read as the pair it comes from: at equal encoder quality the 500-photo WebP atlas costs essentially the same bytes as individual files (-0.2% at q80) but reaches a slightly lower mean per-tile SSIM (0.975 vs 0.978, a 0.004 deficit); the matched-quality protocol prices that small deficit as the -8.5% through interpolation on a steep rate-distortion curve. The sign is robust (the deduplicated-subset bootstrap interval is $[-12.0, -6.7]$), but the magnitude is metric-dependent, and Section 5.5 shows two encoder flags reverse it to $+5\%$. Edge-replicated padding costs roughly 7 percentage points of saving per 8-pixel step for JPEG and roughly 10 for WebP (flat art, $N = 200$), pricing the block-alignment mitigations against chroma bleed.

The absolute comparison compounds codec choice with bundling: at $N = 500$ and SSIM 0.97 on flat art, individual JPEG files cost 630 KB, the JPEG atlas 464 KB, individual WebP files 240 KB, and the WebP atlas 220 KB. Moving a legacy individual-JPEG deployment to a WebP atlas cuts bytes by 65%; the format change contributes most of it, and bundling contributes the rest while also collapsing 500 requests into one.

Matched-quality comparison equalizes the mean, not the tails. Table 2 shows the per-tile SSIM distribution: the WebP atlas systematically carries a worse worst tile than individual files (a flat-art

tile stuck at 0.85 that no quality setting recovers), while JPEG's atlas and individual tails coincide. A practitioner enforcing a hard per-tile floor rather than a mean should treat the WebP pixel atlas accordingly, or use the byte-bundle, which preserves each tile's own encoding.

Table 2. Per-tile SSIM distribution (mean, 5th percentile, minimum) for individual files versus the grid atlas at equal encoder quality (q80, N=500). Matching the mean can still leave the atlas with a worse worst tile: the flat-art WebP atlas has a tile stuck near 0.85 that no quality setting recovers, a shared-segment boundary casualty, whereas JPEG's atlas and individual tails coincide.

condition	ind mean	ind p5	ind min	atlas mean	atlas p5	atlas min
flat art 72px, WebP	0.991	0.984	0.954	0.984	0.960	0.853
flat art 72px, JPEG	0.979	0.967	0.946	0.979	0.967	0.957
photos 56px, WebP	0.979	0.961	0.925	0.980	0.963	0.918
photos 224px, WebP	0.978	0.965	0.942	0.975	0.960	0.926

The crossover is not an artifact of the luma-SSIM metric. We re-measured the photo savings with SSIMULACRA2, the reference libjxl perceptual metric, scoring each condition on the full grid image at matched quality (Table 3). JPEG's atlas saving stays positive at every tile size (about +33% at 56 px, +12% at 112 px, and +3% at 224 px at a matched score of 70), tracking the luma-SSIM result closely. WebP is positive only for the smallest tiles (+12% at 56 px) and turns clearly negative by 112 px (-24%) and 224 px (-29%). The break-even therefore survives the change of metric and moves to a smaller tile, and the WebP penalty is larger under SSIMULACRA2 than under luma SSIM (-29% vs -8.5% at 224 px): the diagnostic that at equal encoder quality the WebP atlas reaches a lower score than individual files while JPEG does not (for example WebP 112 px q80 scores 72.9 as an atlas against 78.0 individually, JPEG 77.2 against 77.3) confirms that the deficit is the shared-quantizer adaptive-state penalty, and that a perceptual metric weighting chroma and blocking prices it higher.

Table 3. Robustness of the photo crossover under a modern perceptual metric: bytes saved by atlasing (%) at matched SSIMULACRA2 (reference libjxl implementation), for 100 photographic tiles scored on the full grid image at two quality targets. JPEG stays positive at every size; WebP is positive only at 56 px and turns clearly negative by 112 px. The direction matches the luma-SSIM result of Table 1, and the WebP penalty is larger under SSIMULACRA2, which weights the chroma and blocking artifacts of VP8's shared quantizer more heavily. Negative values (red) mean the atlas is larger.

class	JPEG @60	JPEG @70	WebP @60	WebP @70
photos 56px	42.2	33.4	2.8	11.9
photos 112px	17.1	12.5	-21.8	-24.3
photos 224px	4.0	3.0	-28.2	-29.4

5.2 Ordering, duplicates, and layout

We re-encode identical tile sets under eight within-atlas orders (source order, three random shuffles, luminance-sorted, mean-color-sorted, k-means cluster-grouped, greedy nearest-neighbor) and two 4-way partitions (random split vs cluster-pure split), for every in-scope codec at N = 500. Three calibrated rules emerge. For lossy codecs, ordering is worth at most one percent: the placement of tiles does not recover the shared-model penalty, so the heuristic can ignore order for JPEG and lossy

WebP and spend its freedom elsewhere. For lossless flat art, order and partition are real levers: source order (which groups semantically related icons) is already near-optimal, random shuffling costs 2–4%, and cluster-pure chunks beat randomly-split chunks by 3 percentage points (lossless WebP, PNG alike), so same-family tiles belong in the same bundle. For PNG specifically, layout is decisive: packing tiles as a one-tile-wide vertical strip lets the per-scanline filters re-adapt at every tile boundary, turning the PNG flat-art atlas from 5.7% larger than individual files (grid layout) to 8.7% smaller, and lifting the photo atlas from –5.2% (grid, deduplicated 224px photos) to parity. Strip packing is therefore the PNG packing rule (chunked to respect browser image-dimension limits), while block-aligned cells are not worth adopting for any codec: aligning 72-pixel tiles to 80-pixel cells costs 7% in bytes for a quality gain below measurement noise.

The largest partition-level effect belongs to collections containing repeated tiles. Real product grids repeat thumbnails freely (variant images, placeholder art, re-listed items); in a test set where 21.6% of tiles were exact repeats, a similarity-sorted atlas cut PNG bytes by 17–18% and lossless WebP bytes by 15–16% relative to unsorted packing, turning both formats' atlas comparison against individual files from negative to clearly positive (+16.9% and +11.5%). The mechanism is windowed matching: sorting places copies within the codec's LZ window, which dedupes them. Exactly-identical tiles are already handled on the web by content-addressed URLs (the browser fetches one copy and many elements reuse it), so the atlas's unique contribution here is compressing near-duplicates: variant and recolored tiles that share most but not all pixels, which per-URL caching cannot merge. The gain applies precisely to the LZ-class codecs and scales with the duplicate rate, so the heuristic prices it as a measured dup-rate term: deduplicate exactly-repeated tiles at the coordinate-map level (many CSS entries, one atlas region), and sort near-duplicates adjacent so the encoder captures the rest.

5.3 Winning content classes: icons, thumbnails, avatars

The largest bundling wins in the whole study belong to lossless flat art, the content of design-system icon sets, map-marker sprites, and flag pickers. On 200 synthetic flat icons with a small shared palette and alpha (Table 4), a WebP-lossless vertical-strip atlas is the smallest option at every size, 42–68% below individual files and **7.6x smaller than the matched-quality JPEG atlas**. A lossless bundle beats the best lossy one outright, because JPEG must spend heavily to avoid ringing on hard edges while the lossless codecs are both smaller and pixel-exact. A shared-palette PNG (one pooled palette across the strip, verified byte-identical after decode) wins 90–97% over individual paletted PNGs when the pooled palette fits, and remains 4.9x smaller than the JPEG atlas. Strip layout beats grid for WebP-lossless at both sizes, and the duplicate-heavy corpus widens the WebP-lossless strip win from 60% to 68% as the LZ window folds repeats. Two boundaries scope this result: the fixed 12-color corpus is the ideal case for shared palettes, so anti-aliased real icons with hundreds of colors will see a smaller palette win (WebP-lossless, which does not depend on palette size, is the robust default there); and for these alpha-bearing assets JPEG is structurally disqualified, so the operative comparison is PNG versus WebP, both of which the strip atlas improves.

Table 4. Bundle bytes for 200 synthetic flat icons (fixed 12-color palette, alpha), lossless unless noted, on a clean corpus and one with 20% exact + 20% near-duplicate tiles. Lossless bundles are byte-exact (palette conversion verified identical after decode); the JPEG row is the matched-SSIM-0.97 grid atlas. Smaller is better.

bundle	clean 24px	clean 48px	dup 24px	dup 48px
individual WebP-lossless files (baseline)	17,328	23,536	17,050	23,007

bundle	clean 24px	clean 48px	dup 24px	dup 48px
WebP-lossless strip	6,966	13,580	5,422	10,606
PNG shared-palette strip	10,900	23,089	6,750	14,855
PNG strip (RGBA)	18,636	38,598	17,802	37,906
WebP-lossless grid	7,780	15,290	7,618	14,910
JPEG atlas (matched SSIM 0.97)	53,266	122,387	51,953	117,377

Two further use cases confirm where WebP wins on realistic content. Product-catalog thumbnails, which sit on white backgrounds, bundle far better than the generic full-frame photos of Section 5.1: at equal WebP quality (per-tile SSIM matched within 0.006) a 100-thumbnail atlas saves 17.8% at 48 px, 14.0% at 64 px, and still 6.7% at 112 px, all positive, because the homogeneous white field nearly removes the shared-segment penalty that drives generic photos negative. Avatar walls add the duplicate dimension: a comment thread of 200 slots drawn from a heavy-tailed popularity distribution resolves to about 45 unique faces, and an atlas of the uniques (with many coordinate-map entries pointing at repeats) saves 26% under WebP and 50% under JPEG at 32 px versus serving the unique files individually. The lossy codecs do not recover the duplication on their own, an atlas of all 200 slots is 5.0x the size of the unique atlas, so deduplication must be explicit at the coordinate-map level. A methodological note accompanies these: when the atlas and individual arms reach nearly identical per-tile quality, as they do on homogeneous thumbnails, matched-SSIM byte interpolation becomes unstable and equal-quality byte comparison is the reliable measurement.

5.4 Encoder parameters

The photographic WebP penalty is not structural. VP8's default encode shares one set of entropy tables and at most four quantizer segments across the whole atlas, and no parameter raises the four-segment ceiling. But two efficiency controls, spatial noise shaping at full strength and adaptive deblocking (autofilter), together move a 120-tile 224-pixel photo atlas from -9.0% to **+5.0%** against individual files at matched SSIM. An equal-quality check isolates the mechanism: with the advanced settings the atlas reaches per-tile SSIM 0.980, above the individual files' 0.979, while encoding 2.4% fewer bytes, because adaptive deblocking removes the block artifacts that the atlas's internal tile-grid boundaries introduce and noise shaping exploits the larger frame's masking context. The knobs improve both arms, but they help the atlas more, so the earlier negative for WebP photographs holds only for the default encoder, not for a tuned one. These controls are reachable through libwebp's configuration (the `cwebp -sns 100 -pass 10 -af` equivalents) though not through every image library's default binding.

5.5 Supporting measurements: network timing

The byte and construction results above are protocol-independent. This section reports how the byte savings translate to load time on a single-testbed browser study; it is supporting evidence for the deployment guidance, not a protocol-measurement contribution, and its absolute timings are environment-dependent.

Table 5. Median time to all 500 tiles visible (ms) per serving condition, protocol, and network profile (n = 11 cold browser loads per cell; 12 run in randomized order, first dropped; WebP payloads). The atl-x column gives the single-atlas speedup over individual files with a 95% bootstrap confidence interval; bun-x is the byte-bundle speedup. Per-cell quartiles are in the released data.

class	network	proto	individual	atlas	atlas x4	byte- bundle	atl-x	bun- x
flat art	localhost	h1	565	126	128	301	4.5x [3.9,5.4]	1.9x
flat art	localhost	h2	448	126	129	319	3.5x [2.3,3.9]	1.4x
flat art	localhost	h3	597	120	135	294	5.0x [4.5,5.7]	2.0x
flat art	100 Mbit / 20 ms	h1	2,074	241	254	466	8.6x [8.5,8.9]	4.4x
flat art	100 Mbit / 20 ms	h2	469	213	216	467	2.2x [2.0,2.3]	1.0x
flat art	100 Mbit / 20 ms	h3	1,579	205	198	431	7.7x [7.0,8.1]	3.7x
flat art	9 Mbit / 60 ms	h1	5,624	790	790	997	7.1x [7.1,7.2]	5.6x
flat art	9 Mbit / 60 ms	h2	790	730	713	1,039	1.1x [1.1,1.1]	0.8x
flat art	9 Mbit / 60 ms	h3	4,020	708	710	1,022	5.7x [5.5,5.8]	3.9x
flat art	9 Mbit / 60 ms / 1% loss	h1	5,827	829	822	1,105	7.0x [6.2,7.5]	5.3x
flat art	9 Mbit / 60 ms / 1% loss	h2	878	832	720	1,098	1.1x [0.7,1.8]	0.8x
flat art	9 Mbit / 60 ms / 1% loss	h3	4,149	867	871	1,189	4.8x [4.7,5.0]	3.5x
pho- tos	localhost	h1	758	424	400	633	1.8x [1.5,1.9]	1.2x
pho- tos	localhost	h2	665	419	422	632	1.6x [1.5,1.6]	1.1x
pho- tos	localhost	h3	835	415	425	662	2.0x [1.7,2.3]	1.3x
pho- tos	100 Mbit / 20 ms	h1	2,019	780	669	1,036	2.6x [2.6,2.6]	1.9x
pho- tos	100 Mbit / 20 ms	h2	692	758	709	1,038	0.9x [0.9,0.9]	0.7x
pho- tos	100 Mbit / 20 ms	h3	2,075	746	763	1,002	2.8x [2.7,2.8]	2.1x
pho- tos	9 Mbit / 60 ms	h1	6,199	3,824	3,897	4,246	1.6x [1.6,1.6]	1.5x
pho- tos	9 Mbit / 60 ms	h2	3,743	3,771	3,866	4,294	1.0x [1.0,1.0]	0.9x

class	network	proto	individual	atlas	atlas x4	byte- bundle	atl-x	bun- x
photo- tos	9 Mbit / 60 ms	h3	6,218	3,902	4,021	4,427	1.6x [1.6,1.6]	1.4x
photo- tos	9 Mbit / 60 ms / 1% loss	h1	6,933	4,612	4,490	7,890	1.5x [1.0,1.7]	0.9x
photo- tos	9 Mbit / 60 ms / 1% loss	h2	5,576	5,993	7,226	6,476	0.9x [0.6,1.3]	0.9x
photo- tos	9 Mbit / 60 ms / 1% loss	h3	8,793	8,404	6,875	7,934	1.0x [0.9,1.3]	1.1x

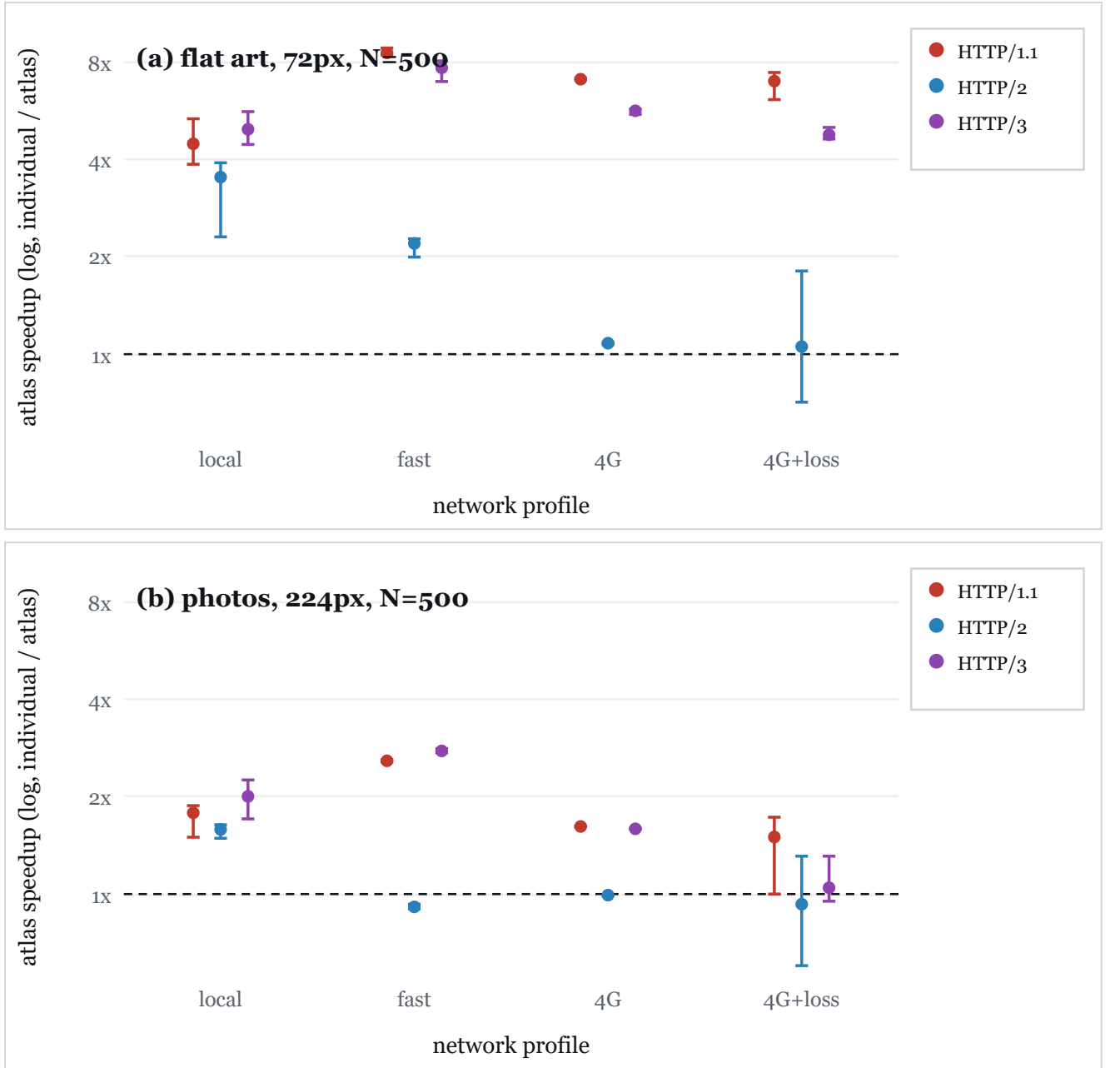


Figure 3. Time-to-all-tiles-visible speedup from atlas (median individual / median single atlas) at $N = 500$, per protocol and network profile, as point estimates with 95% bootstrap confidence intervals on a shared log axis; the dashed line marks parity (1x). HTTP/2 sits near parity for many small files, while HTTP/1.1 and HTTP/3 retain multi-x speedups; intervals that straddle the parity line (notably the loss-dominated 4G+loss profile) mark cells where atlas and individual are statistically indistinguishable.

Three protocol-level facts organize Table 5 and Figure 3. First, on HTTP/1.1 and HTTP/3, bundling remains a large timing win for flat art: 4.5–8.6x and 4.8–7.7x respectively for 500 tiles across the network profiles, and 1.5–2.8x for photos. Across cells, added network impairment never speeds a condition up, and the reported speedups carry tight bootstrap intervals (Table 5). Second, HTTP/2 is the strongest protocol for many small files: its multiplexing loads 500 individual images almost as fast as the atlas on bandwidth-limited links (1.0–1.1x at 9 Mbit), so under HTTP/2 the case for bundling small tiles rests chiefly on the byte saving of Table 1 (up to 26% for flat art) rather than on latency. Third, on this testbed HTTP/3's individual-file loads run 4–5x slower than HTTP/2's on the same links, including at zero loss, and a concurrency diagnostic locates the cause. The diagnostic reconstructs each request's in-flight interval from the Resource Timing API and finds the peak number of simultaneous image requests: HTTP/2 sustained 13–14 at once while HTTP/3 sustained only 6, roughly half the parallelism, and the slowdown tracks that gap. The server's own QUIC transport log (quic-go qlog) confirms the effect and locates it precisely: across cold HTTP/3 loads of the 500-file page, the server records a peak of only 4–6 request streams open at once (median 5 for photographs, 4 for flat art), agreeing with the browser-side count, while its transport parameters advertise a 100-stream limit (`initial_max_streams_bidi`) that is never approached. The constraint is thus not a QUIC stream-limit or flow-control effect, the server permits 100 concurrent streams, but the client's QUIC request scheduler (Chromium's), a property of the deployment, not of HTTP/3, which multiplexes independent streams over one connection. A default HTTP/3 client can therefore under-multiplex a many-small-image page relative to HTTP/2, which makes bundling valuable there. Under the 1% packet-loss profile every serving condition becomes noise-dominated on this testbed: per-cell coefficients of variation reach 0.4 and the atlas-vs-individual and chunk-vs-atlas differences fall inside overlapping confidence intervals (for example lossy4g photos on HTTP/1.1 give a single atlas at 1.50x and four chunks at 1.54x, indistinguishable). We therefore make no loss-recovery claim for chunking from the timing data; chunking's demonstrated benefit is cache granularity and bounded invalidation (Section 6.2, Figure 4), not packet-loss resilience. The byte-bundle beats individual serving nearly everywhere on HTTP/1.1 and HTTP/3 (up to 5.7x) at exactly the individual conditions' byte cost, which makes it the bundling method of choice for content whose pixels should not share a codec model (photos, lossless assets); the pixel atlas remains faster where its byte savings compound with the request savings.

6 Discussion

6.1 The coupling spectrum

The results organize on a single axis: what a set of tiles is made to share. At one end, the byte-bundle shares only transport: it concatenates independently-encoded files, so it never costs bytes and never gains cross-image compression. Two standard-container formats sit at the same point, JPEG abbreviated format (one shared table datastream plus per-tile table-less scans) and animated WebP (one independent payload per frame), both measured byte-equivalent to the byte-bundle while offering a playable container or per-tile random access. A shared-table bundle adds table sharing; the pixel atlas goes furthest, sharing the entire coding model, palette, entropy context, quantizer segments, and filters. The measurements then reduce to one principle: bundling *saves* to the extent tiles share *fixed* costs (container overhead, tables, duplicate content), and *loses* to the extent they are forced to share *adaptive* state (a single quantizer allocation, one scanline-filter prediction context, one palette) that

individually-encoded files would have specialized. The fixed-cost half of this account is directly testable: regressing each codec's measured saving (47 matched-quality cells) on the overhead-to-content predictor $100 \cdot H \cdot N / \text{bytes}_{\text{individual}}$ gives $R^2 = 0.99$ for JPEG, where the container tables dominate, so amortization alone explains the JPEG savings almost completely (slope 0.59, i.e. the 600-byte figure overstates the recoverable share). For lossy WebP the same predictor captures only the ordering ($R^2 = 0.69$) and badly under-predicts the magnitude, the measured range of -8.5 to $+19\%$ dwarfs the predicted 0.5 to 6%, and for the lossless formats the fixed-cost term fails outright ($R^2 < 0.2$, negative for lossless WebP). Those are precisely the codecs where the adaptive-state term dominates: the shared four-segment allocation for WebP, the per-scanline filter for PNG. The coupling account is thus a two-term decomposition, a fixed-cost gain that is quantitatively predictive for JPEG and an adaptive-state penalty that is codec-specific and, at present, characterized rather than modeled; JPEG always wins because its shared cost is large and its adaptive coupling weak, lossless flat art wins because a shared palette is pure fixed-cost saving, and photographic WebP loses under the default encoder because the shared segments are adaptive state, recovering once adaptive deblocking neutralizes the artifact.

A first step toward modeling the second term rather than only naming it: adding a content-heterogeneity feature (the mean pairwise distance of tile thumbnails) with a per-codec penalty coefficient lifts a pooled cross-codec fit from $R^2 = 0.73$ (fixed-cost term alone) to 0.78, and the learned penalties order the codecs by their adaptive coupling exactly as the mechanism predicts (JPEG 1.2, PNG 5.5, lossless WebP 12.7, lossy WebP 21.0). The modest lift shows a generic heterogeneity proxy captures the direction but not the full magnitude of the penalty; a predictive model accurate across codecs will need codec-specific structure (the four-segment allocation for VP8, the per-scanline filter for PNG), which we leave to future work. The practical takeaway stands without the full model: measure the two candidate representations for a group and keep the smaller, which is what the heuristic of Section 6.3 does and why it reaches the oracle out-of-sample.

The spectrum has a hard end. Individual serving costs $\Sigma_i (H_i + C_i(x_i))$, where H_i is a file's fixed overhead and C_i its compressed payload under state tuned to that image; an atlas costs $H + C(x_1..x_N; \theta)$ with one shared header and one shared adaptive state θ . Bundling wins only when the saved overhead $\Sigma_i H_i - H$ exceeds the cost of replacing each image's optimal state by the shared θ . For large, unrelated photographs $H_i/C_i \rightarrow 0$: almost no fixed overhead remains to amortize while any adaptation penalty persists, so a single browser-decodable pixel atlas is not expected to beat independently-optimized files for a sufficiently large, heterogeneous photographic corpus. This is the boundary the measurements trace, JPEG's advantage falling from $+30\%$ at 56 px to $+3\%$ at 224 px, and it is why the byte-bundle, which shares only transport, is the principled zero-coupling endpoint rather than a fallback. The open direction is to decouple adaptation spatially while keeping single-resource amortization: WebP's lossless format already carries a spatially-varying entropy image (meta-prefix groups), and aligning that grid to atlas tile boundaries, which today requires instrumenting the encoder, is the most promising route to a lossless photographic atlas that wins on bytes.

6.2 Cache invalidation and delta updates

Bundling's one structural liability is cache granularity: editing a single tile invalidates the whole bundle, so a naive re-deploy re-downloads everything. Formally, a partition into bundles $B_1..B_k$ with per-tile update probabilities p_i pays expected bytes $\Sigma_j [1 - \prod_{i \in B_j} (1 - p_i)] \cdot \text{bytes}(B_j)$ per deploy, which

grows with bundle size and with churn. Chunking bounds it at $1/k$ of the collection; grouping tiles by update cadence confines a change to the chunk that carries it. The sharpest remedy is to serve updates as deltas against the cached previous version (Compression Dictionary Transport). In a 392-photo WebP bundle, a zstd delta against the prior bundle cost 25 KB when 5 tiles changed and 237 KB when 50 changed, tracking the changed-files optimum and beating whole-atlas re-download (2.8 MB) by two orders of magnitude. The structural weakness thus becomes proportional to churn rather than to bundle size.

A warm-cache simulation over a 200-tile WebP collection makes the full trade-off explicit (Figure 4). Serving immutable individual files is the granularity ideal, a returning client fetches only the changed tiles (12 KB at 1% churn). A single monolithic atlas is the opposite extreme: any change re-downloads the entire 1.47 MB, 126x the individual cost at 1% churn. Chunking closes the gap proportionally, 16 chunks cost 170 KB at 1% churn against one atlas's 1.47 MB, but never reaches per-tile granularity. Dictionary-delta serving does: at 12 KB for 1% churn it matches the individual-file optimum within one percent, and at 20% churn it undercuts it (192 vs 218 KB) because the shared dictionary compresses each new tile against the old ones. The practical consequence is that an update-heavy collection can keep the cold-load benefits of bundling and still get individual-file cache granularity, by serving deltas rather than whole bundles.

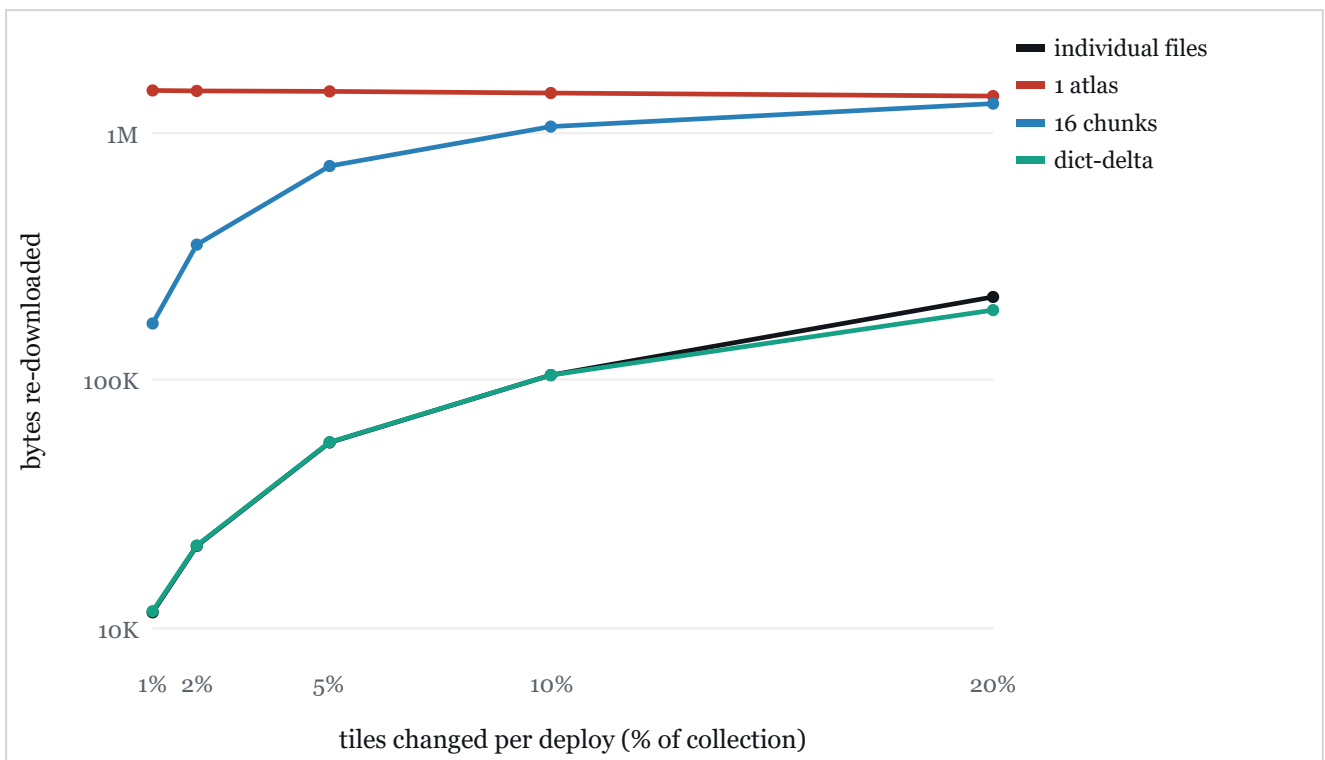


Figure 4. Warm-cache re-download: bytes a returning client fetches after a deploy changes a fraction of a 200-tile WebP collection (log scale; full fresh download 1,474,036 B; mean of 20 random deploys per rate). A single atlas re-fetches almost everything on any change; 16 chunks scale with churn; dictionary-delta serving tracks the individual-file optimum and undercuts it at high churn.

6.3 A calibrated construction heuristic

The cost above is not minimized by search; instead the measured curves calibrate a greedy heuristic, implemented as a command-line tool (`atlas_optimizer`, released with the study). The tool folds exact duplicates into shared coordinates; partitions tiles by update cadence, lossless requirement, and dimensions; routes each group to a pixel atlas (small lossy tiles), a byte-bundle (large or lossless tiles),

or individual files (tiny groups); and chunks each bundle (Figure 5). It emits the atlas and bundle files, a CSS coordinate map, a loader snippet, and a per-group savings report. Validated against the study's own asset sets, it converts 521 icon-class tiles into four atlas chunks at 19% fewer bytes, and 521 photo thumbnails, of which 21.6% are exact repeats, into four byte-bundle chunks at 21% fewer bytes, with 521 requests becoming four in both cases.

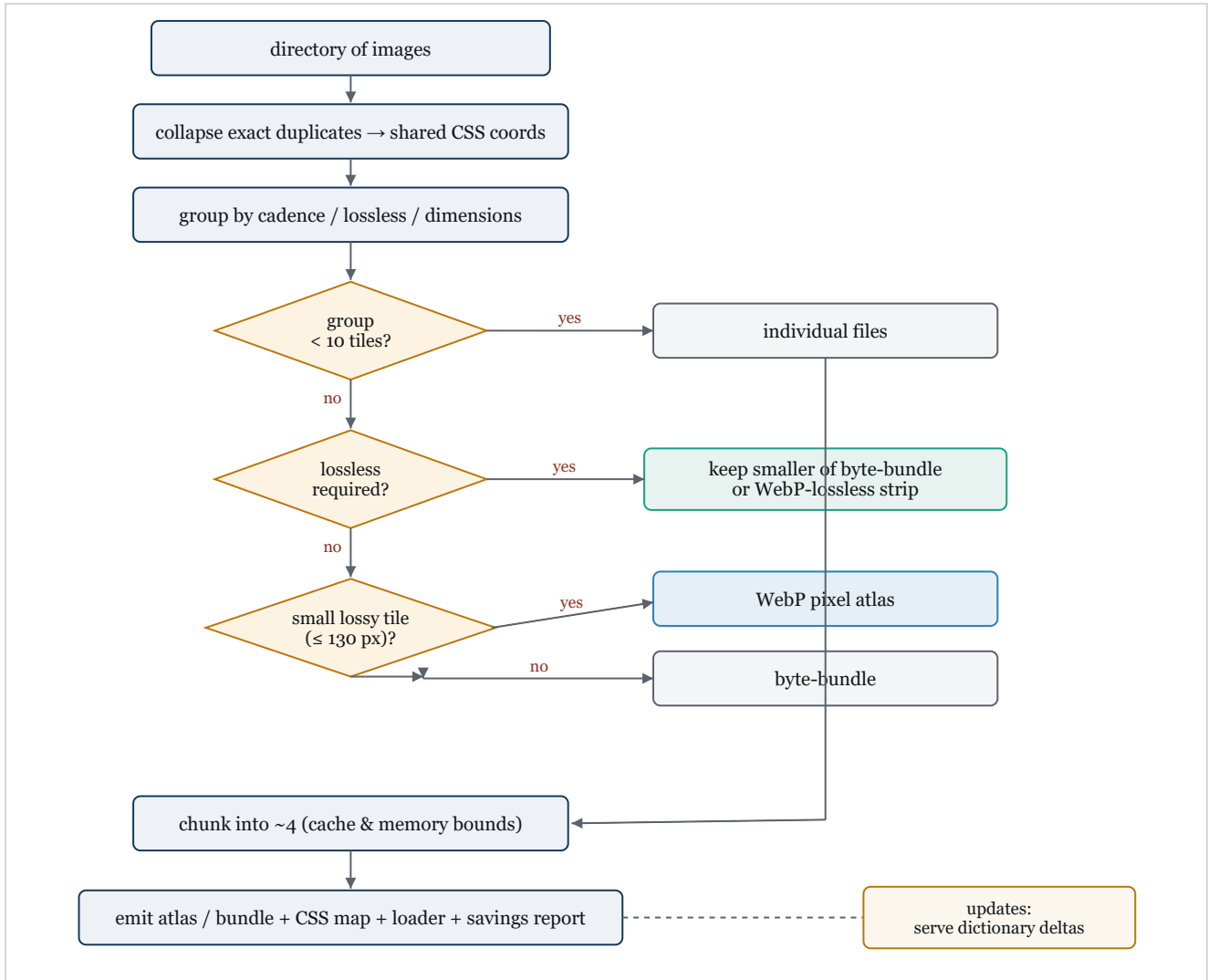


Figure 5. The construction heuristic as implemented in `atlas_optimizer`. Exact duplicates collapse to shared coordinates; tiles are grouped by update cadence, lossless requirement, and dimensions; each group is routed by a short decision cascade to individual files, a pixel atlas, or a byte-bundle, with a measured tiebreak for lossless groups (keep the smaller of a byte-bundle and a WebP-lossless strip). Groups are chunked for bounded cache invalidation and memory, and the tool emits the atlas and bundle files, a CSS coordinate map, a loader, and a savings report; piecewise updates can be served as dictionary deltas. This is the procedure evaluated against the oracle in Table 6.

Because those asset sets also shaped the rules, a fair test requires collections the heuristic never saw. We evaluate it on five independent corpora, Noto emoji, OpenMoji, country flags, Flickr photographs, and Robohash avatars, none of which were used in calibration, against an offline oracle that enumerates the candidate configurations and returns the byte-optimal one (Table 6). Each group carries one measured tiebreak: for a lossless group the heuristic keeps the smaller of a byte-bundle and a WebP-lossless strip, since neither dominates (a strip wins on OpenMoji and Robohash, a byte-bundle on Noto). With that tiebreak the heuristic's automatic choice equals the oracle on all five corpora (0% regret), including the two content types, generated avatars and a second emoji vendor, that most differ

from the calibration sets. This is the property that matters for a deployable tool: on collections it was not built from, it does not merely save bytes, it chooses at or near the best available configuration.

The value is in choosing, not in any single layout. Table 6 also scores three fixed rules a developer might hardcode, each still allowed the best admissible codec: always build one pixel atlas, always byte-bundle, or always strip-pack. Every one is beaten on at least one corpus (mean regret 5.8%, 9.4%, and 186%; always-strip alone costs +643% on photographs, where a lossless strip is disastrous), because the byte-optimal layout flips with content: strips win on the sparse OpenMoji and Robohash sets, a byte-bundle wins on dense Noto and on photographs, a pixel atlas wins on the flat-color flags. The calibrated heuristic tracks those flips to zero regret; no fixed rule does.

Table 6. Out-of-sample evaluation of the construction heuristic on five independent corpora that played no role in its calibration (Noto emoji, OpenMoji, country flags, Flickr photos, Robohash avatars; 100 tiles each). An offline oracle enumerates every candidate configuration (individual, grid atlas, strip atlas, byte-bundle across the admissible codecs) at matched quality and reports the byte-optimal one; regret is a policy’s bytes over the oracle’s. The last three columns are simple fixed-layout rules a developer might hardcode, each still allowed the best admissible codec (individual files cost the same bytes as a byte-bundle). The calibrated heuristic matches the oracle on all five corpora; every fixed rule is beaten on at least one, because the winning layout is content-dependent. Smaller regret is better.

corpus	class	tiles	heuristic choice	heuristic	always atlas	always bundle	always strip
noto	flat-art	100	byte-bundle(webpll)	0.0%	+14.3%	0.0%	+5.1%
open-moji	flat-art	100	strip-atlas(webpll)	0.0%	+3.7%	+6.4%	0.0%
flags	flat-limited	100	pixel-atlas(webp)	0.0%	0.0%	+2.3%	+281.2%
flickr	photo	100	byte-bundle(webp)	0.0%	+6.8%	0.0%	+643.0%
robo	avatar	100	strip-atlas(webpll)	0.0%	+4.5%	+38.1%	0.0%
<i>mean regret</i>				0.0%	+5.8%	+9.4%	+185.9%

6.4 Limitations

Quality is measured by luma SSIM at a 0.97 target; the photo crossover is confirmed under the SSIMULACRA2 perceptual metric (Section 5.1, Table 3), and applying it or butteraugli across the full sweep would further sharpen the matched-quality comparison. The shared-palette result uses a synthetic limited-color corpus that is the ideal case; anti-aliased production icons will realize less of it. The network study emulates four profiles on a single-machine testbed rather than the open Internet, and measures time-to-all-tiles-visible rather than a full field-metric suite. Decoded-memory cost is bounded analytically and by the chunking recommendation but not yet profiled in a live renderer. The HTTP/3 concurrency diagnosis is confirmed from the server’s own QUIC transport log (Section 5.5); testing its generality across a second QUIC implementation and native Linux remains future work. The matched-quality crossovers are calibrated on one icon set and one photographic collection with randomized-subset bootstrap intervals (Section 5.1), and the heuristic is validated out-of-sample on five further independent corpora (Section 6.3); a still wider survey of naturally-occurring collections would further generalize the reported thresholds. The network numbers are medians of 11 cold loads per cell with bootstrap confidence intervals (Table 5) but no formal significance testing, and the timing endpoint is time-to-all-tiles-visible; user-centric metrics (first tile, above-the-fold completion,

LCP, decode CPU) and warm-cache multi-navigation behavior under realistic asset churn are not measured and could change the recommended bundle size. Finally, AVIF and JPEG XL are excluded by scope, though AVIF is now a material share of web imagery; their larger container overhead and richer adaptivity would move the crossovers, and AVIF is the first codec a follow-up should add.

7 Conclusion

Bundling small web images pays, and the study makes precise when and by how much for the three formats that carry most of the web's images. Atlas small lossy tiles: icons and thumbnails gain up to 30% under JPEG and up to 19% under WebP at matched quality, and lossless flat art, the icon-set and map-marker case, gains 40–95% with a strip-packed or shared-palette bundle that is smaller than even the best lossy option. Serve larger photographs and lossless assets as a byte-bundle, which collapses requests at zero byte cost, or, for photographic WebP, tune the encoder (noise shaping plus adaptive deblocking) to turn the atlas penalty into a gain. Ship about four chunks for bounded cache invalidation and decoded-memory limits, deduplicate repeats explicitly, and serve updates as dictionary deltas. On the wire, bundling is 4.5–8.6x faster to full visibility on HTTP/1.1 and HTTP/3 and chiefly a byte optimization on HTTP/2. The unifying account, that savings come from sharing fixed costs and losses from sharing adaptive state, is quantitatively predictive for JPEG ($R^2 = 0.99$) and organizes the codec-specific behavior of the rest, and it guides the accompanying construction heuristic, which turns a directory of images into deployable bundles.

References

- [1] C. Johnson. Forgo JS packaging? Not so fast. Khan Academy Engineering, 2015. <https://blog.khanacademy.org/forgo-js-packaging-not-so-fast/>
- [2] C. Coyier. Musings on HTTP/2 and bundling. CSS-Tricks. <https://css-tricks.com/musings-on-http2-and-bundling/>
- [3] M. Varvello, K. Schomp, D. Naylor, J. Blackburn, A. Finamore, K. Papagiannaki. Is the Web HTTP/2 Yet? Passive and Active Measurement (PAM), LNCS 9631, 2016. doi:10.1007/978-3-319-30505-9_17
- [4] R. Meireles, J. Liu, P. Steenkiste. A study of HTTP/2's Server Push Performance Potential. arXiv:2207.05885, 2022.
- [5] T. Hunter. HTTP/3 is fast. Request Metrics, 2022. <https://requestmetrics.com/web-performance/http3-is-fast/>
- [6] T. Eden. What's the smallest file size for a 1 pixel image? 2024. <https://shkspr.mobi/blog/2024/01/whats-the-smallest-file-size-for-a-1-pixel-image/>
- [7] J. Sneyers. One pixel is worth three thousand words. Cloudinary Blog. https://cloudinary.com/blog/one_pixel_is_worth_three_thousand_words
- [8] HTTP Archive. Web Almanac 2024, Media chapter. <https://almanac.httparchive.org/en/2024/media>
- [9] Unity Technologies. Sprites.AtlasSettings.paddingPower documentation. <https://docs.unity3d.com/ScriptReference/Sprites.AtlasSettings-paddingPower.html>
- [10] D. Shea. CSS Sprites: Image Slicing's Kiss of Death. A List Apart, 2004. <https://alistapart.com/article/sprites/>
- [11] Google. Web Fundamentals: HTTP/2 and resource bundling guidance. <https://web.dev/articles/http2>
- [12] M. Belshe, R. Peon, M. Thomson. Hypertext Transfer Protocol Version 2 (HTTP/2). RFC 7540, IETF, 2015. doi:10.17487/RFC7540
- [13] M. Bishop. HTTP/3. RFC 9114, IETF, 2022. doi:10.17487/RFC9114
- [14] J. Ruth, D. Kunze, O. Hohlfeld. Measuring HTTP/3: Adoption and Performance. arXiv:2102.12358, 2021.
- [15] U. Goel et al. Domain-Sharding for Faster HTTP/2 in Lossy Cellular Networks. arXiv:1707.05836, 2017.
- [16] Cloudinary. Image optimization documentation (format selection and the <5,000-pixel AVIF policy). https://cloudinary.com/documentation/image_optimization

- [17] ITU-T T.81 / ISO IEC 10918-1. Digital compression and coding of continuous-tone still images (JPEG), 1992.
- [18] W3C. Portable Network Graphics (PNG) Specification, 3rd ed., 2003. <https://www.w3.org/TR/PNG/>
- [19] Google. WebP Container and Bitstream Specification. https://developers.google.com/speed/webp/docs/riff_container
- [20] J. Ratcliff. Texture atlas / sprite packing techniques. Game Developer, 2002.
- [21] J. Jylänki. A Thousand Ways to Pack the Bin: rectangle bin-packing algorithms (skyline, MaxRects), 2010.
- [22] J. Alakuijala, Z. Szabadka. Brotli Compressed Data Format. RFC 7932, IETF, 2016. doi:10.17487/RFC7932
- [23] Y. Collet, M. Kucherawy. Zstandard Compression and the application/zstd Media Type. RFC 8878, IETF, 2021. doi:10.17487/RFC8878
- [24] P. Meenan, Y. Weiss. Compression Dictionary Transport. IETF draft / Chrome Platform Status, 2024. <https://datatracker.ietf.org/doc/draft-ietf-httpbis-compression-dictionary/>
- [25] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE Trans. Image Processing 13(4):600-612, 2004. doi:10.1109/TIP.2003.819861
- [26] X. S. Wang, A. Balasubramanian, A. Krishnamurthy, D. Wetherall. Demystifying Page Load Performance with WProf. USENIX NSDI, 2013.
- [27] X. S. Wang, A. Balasubramanian, A. Krishnamurthy, D. Wetherall. How Speedy is SPDY? USENIX NSDI, 2014.
- [28] R. Netravali, A. Goyal, J. Mickens, H. Balakrishnan. Polaris: Faster Page Loads Using Fine-grained Dependency Tracking. USENIX NSDI, 2016.
- [29] R. Marx, T. Wijnants, P. Quax, A. Faes, W. Lamotte. Concatenation, Embedding and Sharding: Do HTTP/1 Performance Best Practices Make Sense in HTTP/2? WEBIST, 2017.
- [30] C. Sander, I. Kunze, K. Wehrle, J. RÜth. Sharding and HTTP/2 Connection Reuse Revisited: Why Are There Still Redundant Connections? ACM IMC, 2021. doi:10.1145/3487552.3487832
- [31] N. Barman, M. G. Martini. An Evaluation of the Next-Generation Image Coding Standard AVIF. IEEE QoMEX, 2020. doi:10.1109/QoMEX48832.2020.9123131
- [32] B. Lévy, S. Petitjean, N. Ray, J. Maillot. Least Squares Conformal Maps for Automatic Texture Atlas Generation. ACM SIGGRAPH / ACM TOG 21(3), 2002. doi:10.1145/566654.566590
- [33] J. Marszałkowski, J. Mizgajski, D. Mokwa, M. Drozdowski. Analysis and Solution of CSS-Sprite Packing Problem. ACM Trans. Web 10(1), Article 1, 2016. doi:10.1145/2818377
- [34] M. Butkiewicz, H. V. Madhyastha, V. Sekar. Understanding Website Complexity: Measurements, Metrics, and Implications. ACM Internet Measurement Conference (IMC), 2011. doi:10.1145/2068816.2068846
- [35] S. Souders. High Performance Web Sites: Essential Knowledge for Front-End Engineers. O'Reilly Media, 2007. ISBN 978-0-596-52930-7.
- [36] H. de Saxcé, I. Oprescu, Y. Chen. Is HTTP/2 really faster than HTTP/1.1? IEEE INFOCOM Workshops (INFOCOM WKSHPS), 2015, pp. 293-299. doi:10.1109/INFCOMW.2015.7179400
- [37] A. M. Kakhki, S. Jero, D. Choffnes, C. Nita-Rotaru, A. Mislove. Taking a Long Look at QUIC: An Approach for Rigorous Evaluation of Rapidly Evolving Transport Protocols. ACM Internet Measurement Conference (IMC), 2017, pp. 290-303. doi:10.1145/3131365.3131368
- [38] J. Alakuijala, R. van Asseldonk, S. Boukortt, M. Bruse, I.-M. Comşa, M. Firsching, et al. JPEG XL next-generation image compression architecture and coding tools. Proc. SPIE 11137, Applications of Digital Image Processing XLII, 111370K, 2019. doi:10.1117/12.2529237
- [39] Google. WebP Compression Study, 2011. https://developers.google.com/speed/webp/docs/webp_study
- [40] G. Randers-Pehrson. MNG (Multiple-image Network Graphics) Format, Version 1.0, 2001. <http://www.libpng.org/pub/mng/spec/> (APNG is standardized in the W3C PNG Specification, 3rd ed. [18]).
- [41] ISO/IEC 23008-12:2017. Information technology — High efficiency coding and media delivery in heterogeneous environments — Part 12: Image File Format (HEIF).
- [42] J. Mogul, B. Krishnamurthy, F. Douglass, A. Feldmann, Y. Goland, A. van Hoff, D. Hellerstein. Delta Encoding in HTTP. RFC 3229, IETF, 2002. doi:10.17487/RFC3229
- [43] D. Korn, J. MacDonald, J. Mogul, K. Vo. The VCDIFF Generic Differencing and Compression Data Format. RFC 3284, IETF, 2002. doi:10.17487/RFC3284

- [44] J. Butler, W.-H. Lee, B. McQuade, K. Mixter. A Proposal for Shared Dictionary Compression over HTTP (SDCH). IETF Internet-Draft draft-lee-sdch-spec, 2008.
- [45] R. Fielding, M. Nottingham, J. Reschke (Eds.). HTTP Caching. RFC 9111 (STD 98), IETF, 2022. doi:10.17487/RFC9111